

Plutoniumprobe im Tresor

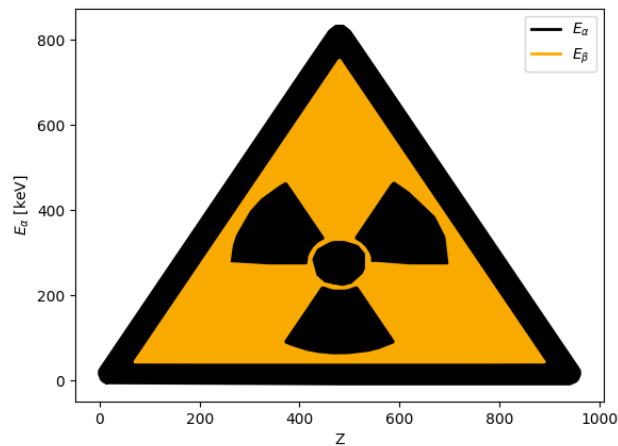


Abb. 0.1: Attention, Attention! Dear comrades! The lab council reports that [...] an unfavorable radiation situation is developing in the lab rooms. [...] ² ³

Auswertung Versuch 251 - Statistik des radioaktiven Zerfalls

Physikalisches Praktikum PAP 2.1

des Studienganges Physik
an der Universität Heidelberg

von

Benjamin Kleijn

11. August 2026

Versuchstermin 07.07.2026

Gruppe, Kurs 9

Tutor:in Nelson Rotenberg

Inhaltsverzeichnis

1. Einleitung	4
1.1. Motivation	4
1.2. Ziele des Versuches	4
1.3. Geräte	4
1.4. Theoretische und technische Grundlagen	6
1.5. Binomialverteilung	6
1.6. Poissonverteilung	6
1.7. Gaussverteilung	7
1.8. Das Geiger-Müller-Zählrohr	7
2. Durchführung	9
2.1. Messverfahren	9
2.2. Messprotokoll	9
3. Auswertung	11
3.1. Zählrohrspannung	12
3.2. Plateaubereich des Zählers	13
3.3. Verifizierung der statistischen Natur radioaktiven Zerfalls	14
4. Diskussion	17
4.1. Plateauanstieg	17
4.2. Messdauer für 1% Genauigkeit	17
4.3. Statistische Natur: Fits	17
4.4. Consistency Check	18
4.5. Fazit	19
Literaturquellen	20
Abbildungsquellen	20
A. Code-Anhang	20

Abbildungsverzeichnis

0.1. <i>Meme</i> : chernobyl evacuation announcement	1
1.1. Versuchsaufbau/Geräte ¹	5
1.2. Poisson- genähert durch Gaußverteilung ¹	7
1.3. Aufbau Geiger-Müller Zählrohr ¹	8
1.4. Plateaubereich des Geiger-Müller ZR ¹	8
3.1. Zählrohrcharakteristik - Spannungskurve	12
3.2. Histogramm und Fit für 2000 Messungen	15
3.3. Histogramm und Fit für 5000 Messungen	15
4.1. useless Version: Abweichungen der Mess- und Fitwerte	18
4.2. χ^2 Summe grafisch dargestellt	19

Tabellenverzeichnis

3.1. Fitergebnisse der statistischen Natur	16
--	----

1. Einleitung

1.1. Motivation

Bei der Arbeit mit radioaktiven Stoffen liegt auf der Hand, ihre Zerfallscharakteristik zu untersuchen: Genau dies wird im folgenden Versuch mit der Neutronenquelle Cobalt-60 ^{60}Co gemacht. Die Zerfallsereignisse werden mit einem Geiger-Müller Zählrohr erfasst, dessen Funktion abhängig der Betriebsspannung eingangs untersucht wird. Anschließend werden statistische Methoden angewandt, die dank Aufnahmen einer hohen Anzahl von Messungen möglich sind, um die Statistik des Zerfalls zu beschreiben. Ein grundlegendes Verständnis statistischer Prozesse ist für die Physik essenziell, um fundierte Vorhersagen über die Verteilung von Messgrößen treffen zu können, denn Messgrößen sind im Allgemeinen nicht deterministisch, können aber mit den mächtigen Werkzeugen der Statistischen Physik beschrieben werden.

1.2. Ziele des Versuches

Der radioaktive Zerfall dient als Beispielexperiment, um ein grundlegendes Verständnis von statistischen Methoden in der Experimentalphysik zu gewinnen. Ziel ist es, die statistischen Verteilungen zu charakterisieren, die dem Zerfallsprozess zugrunde liegen, und die Anwendbarkeit verschiedener Wahrscheinlichkeitsmodelle wie der Poisson- und Gauß-Verteilung zu überprüfen.

1.3. Geräte

Der Aufbau besteht im wesentlichen aus einem Geiger-Müller-Zählrohr, welches auf einen Präparatehalter mit Bleiabschirmung ausgerichtet ist, in den das untersuchte Präparat eingespannt wird. Die Halterung befindet sich auf einer Schiene, sodass der Abstand zum Zählrohr angepasst werden kann. Das Zählrohr ist mit dem zugehörigen Betriebsgerät verbunden, mit welchem sich Einstellungen wie Betriebsspannung vornehmen lassen, und welches die Zählrate an einen externen Impulszähler weitergibt, das wiederum seine Messwerte an einen Computer sendet, der die Messdaten speichert und visuell auswertet (in Form von Histogrammen).

1. Geiger-Müller Zählrohr mit Betriebsgerät
2. externer Impulszähler
3. PC
4. Präparatehalterung mit Bleiabschirmung
5. Radioaktives Präparat ^{60}Co

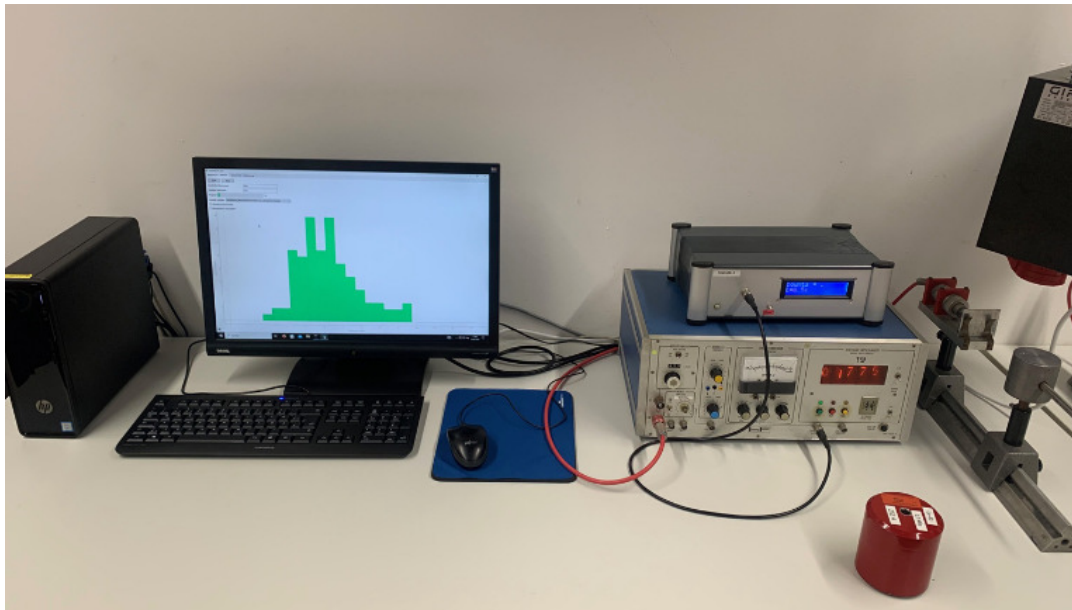


Abb. 1.1: Versuchsaufbau/Geräte¹

1.4. Theoretische und technische Grundlagen¹

Radiaktivität ist ein Prozess, bei dem durch Kernreaktionen ionisierende Strahlung ausgesendet wird. Man unterscheidet verschiedene Strahlungsarten anhand der jeweils emittierten Teilchen: Bei α -Strahlung werden Heliumkerne, bei β -Strahlung Elektronen (oder Positronen beim β^+ -Zerfall) und bei γ -Strahlung Photonen ausgesendet. Wann ein einzelner Zerfall stattfindet, ist nicht vorhersagbar, prinzipiell handelt es sich beim Zerfall eines radioaktiven Isotops jedoch um ein Ereignis mit zwei möglichen Ausgängen: Entweder ein radioaktiver Atomkern zerfällt mit der Wahrscheinlichkeit p innerhalb eines gewissen Beobachtungszeitraums oder nicht. Die Zerfallswahrscheinlichkeit hängt von der Beobachtungsdauer gemäß $p(t) = 1 - e^{-\lambda t}$ ab. Ist die Zerfallskonstante λ sehr klein, wie es bei ^{60}Co der Fall ist, so kann p für einen festen Beobachtungszeitraum als konstant angenommen werden.

Damit lassen sich nun statistische Aussagen nutzen, um die Wahrscheinlichkeitsverteilung von k Zerfällen der Wahrscheinlichkeit p bei n Atomkernen in der Zeit t zu beschreiben.

1.5. Binomialverteilung

Die Binomialverteilung beschreibt ein diskretes Zufallsexperiment mit genau zwei möglichen Ergebnissen. Führt man ein solches Experiment n -mal durch, wobei die jeweiligen Durchführungen unabhängig voneinander sind, so ist die Wahrscheinlichkeit B , k -mal das erste Ergebnis zu erhalten, binomialverteilt mit den Parametern n und p :

$$B_{n,p}(k) = \binom{n}{k} p^k (1-p)^{n-k} \quad (1.1)$$

Die Binomialverteilung ist im Zusammenhang mit dem radioaktiven Zerfall relevant, da beim Betrachten von n Kernen im Zeitintervall $(0, t)$ die Anzahl der Zerfälle k $B_{n,p}(t)$ -verteilt ist.

1.6. Poissonverteilung

Betrachtet man große n und kleine p , so lässt sich die Binomialverteilung $B_{n,p}$ durch die Poissonverteilung P_λ nähern:

$$P_\lambda(k) = e^{-\lambda} \frac{\lambda^k}{k!} \quad (1.2)$$

Dabei ist der Parameter λ der Poissonverteilung näherungsweise gegeben durch $\lambda = np$. Diese Verteilung beschreibt den radioaktiven Zerfall gut, da man in der Regel viele Kerne (großes n) und kleine Zerfallswahrscheinlichkeiten betrachtet. Die Kenngrößen der Poissonverteilung sind ihr Erwartungswert $\mu = \lambda$ und ihre Standardabweichung $\sigma = \sqrt{\lambda}$.

1.7. Gaussverteilung

Die Gaußverteilung spielt eine entscheidende Rolle in der Wahrscheinlichkeitstheorie und Statistik. Ihre Dichte ist gegeben durch

$$G_{\mu,\sigma}(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}} \quad (1.3)$$

Die Verteilung ist eindeutig bestimmt durch ihren Erwartungswert μ und ihre Standardabweichung σ . Im Zusammenhang mit dem radioaktiven Zerfall ist relevant, dass sich die Poissonverteilung für große λ durch eine Gaußverteilung mit $\mu = \sigma^2 = \lambda$ annähern lässt.

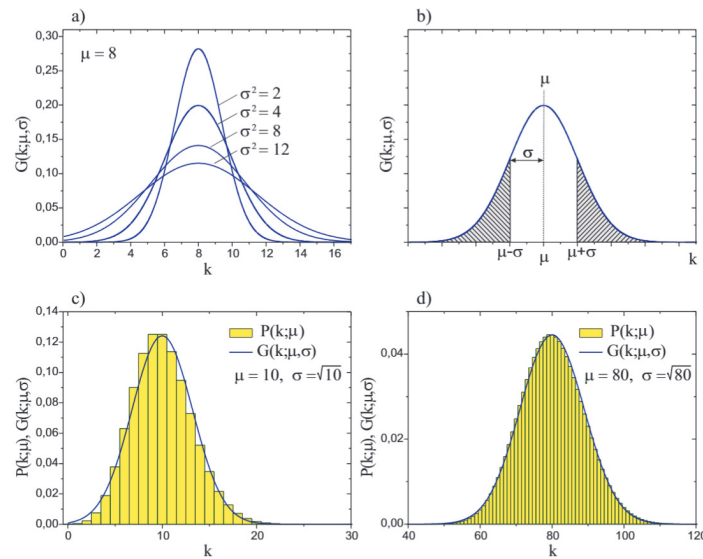


Abb. 1.2: Darstellung der Annäherung der Poissonverteilung an die Gaussverteilung¹

1.8. Das Geiger-Müller-Zählrohr

Das Geiger-Müller-Zählrohr dient als Messgerät für ionisierende Strahlung. Sein Funktionsprinzip basiert auf der Ionisation von Gasatomen innerhalb des Rohrs: Trifft ein ionisierendes Teilchen auf das Füllgas, werden Gasteilchen ionisiert. Die dabei entstehenden freien Elektronen werden durch die anliegende Hochspannung beschleunigt und können durch Stoßionisation weitere Gasatome ionisieren. Dieser Prozess führt zu einer schnellen Vermehrung der Ladungsträger, einer sogenannten Elektronenlawine. Die Lawine erzeugt einen messbaren Strompuls, der verstärkt und von einem Zähler detektiert werden kann. Um sicherzustellen, dass die Elektronenlawine nach jedem Detektionsereignis wieder abklingt und das Zählrohr für das nächste Teilchen bereit ist, wird dem Füllgas ein sogenanntes Löschgas beigemischt.

Die Anzahl der ionisierten Gasatome und damit die Amplitude des erzeugten Strompulses hängen von der Betriebsspannung des Zählrohrs ab. Da energiereichere einfallende Teilchen

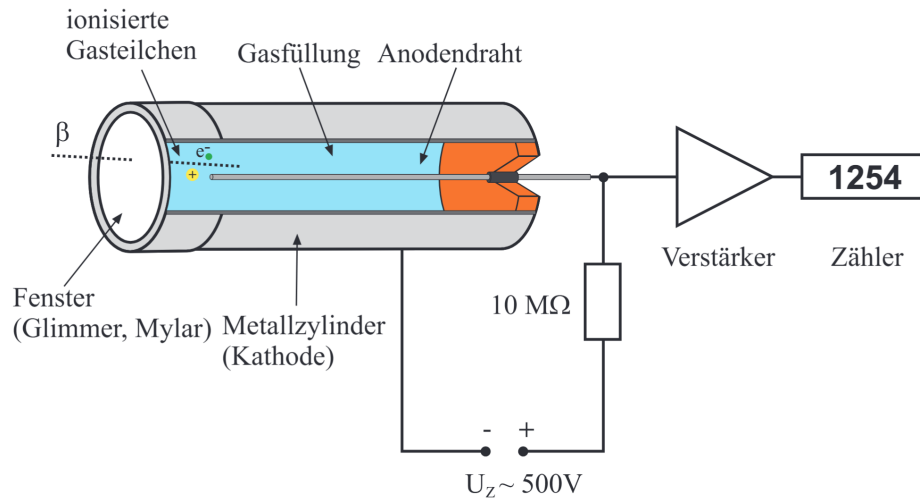


Abb. 1.3: Schematischer Aufbau des Geiger-Müller-Zählrohres¹

prinzipiell mehr Gasatome ionisieren, ließe sich theoretisch auf die Energie der Teilchen schließen. Für den vorliegenden Versuch ist jedoch ausschließlich die Bestimmung der Zerfallsanzahl von Interesse. Daher muss das Zählverhalten unabhängig von der Energie der einfallenden Teilchen sein. Dies wird erreicht, indem die Betriebsspannung so eingestellt wird, dass sich das Zählrohr im Plateaubereich seiner Kennlinie befindet. In diesem Bereich ist die Zählfrequenz weitgehend konstant und unempfindlich gegenüber Schwankungen der Spannung oder der Teilchenenergie.

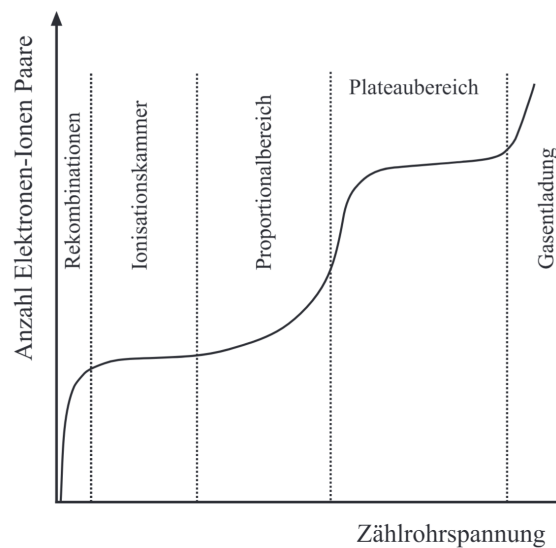


Abb. 1.4: Plateaubereich des Geiger-Müller-Zählrohres¹

2. Durchführung

2.1. Messverfahren

Aufgabe 1: Skizze des Versuchsaufbaus im Messprotokoll.

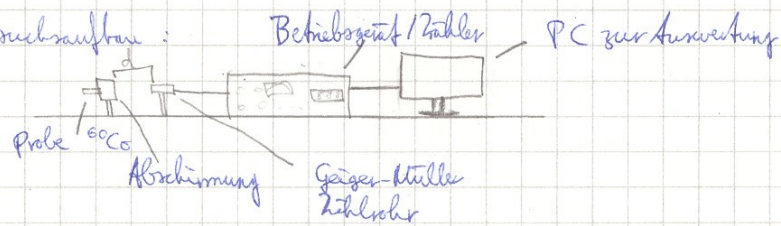
Aufgabe 2: Inbetriebnahme des GM-ZR: Mithilfe der akustischen Ausgabe der Zerfallsdetektion wird die Einsatzspannung U_0 (schlagartiges Auftauchen des akustischen Signals) bestimmt und anschließend bei einer Totzeit von $t = 30$ s und schrittweiser Spannungserhöhung die Ereigniszahl N bestimmt.

Aufgabe 3: Plateauanstieg: Das Präparat wird so nahe wie möglich an das Zählrohr gestellt und für U_0 sowie $U_0 + 100$ V für (60, 180) s die Ereigniszahl gemessen.

Aufgabe 4&5: 2000, 5000 Messungen der Zerfallszahl pro $t = (0.5, 0.1)$ s, die in ein Histogramm aufgetragen werden. Bei der ersten Messung wurde die Zerfallsrate auf $n = (150 \text{ bis } 160) \text{ s}^{-1}$, bei der zweiten auf $n = (40 \text{ bis } 50) \text{ s}^{-1}$ eingestellt. Dies bewirkt einen geringen Mittelwert von gemessenen Ereignissen pro Zeit t bei der zweiten Verteilung.

2.2. Messprotokoll

Skizze Versuchsaufbau:



Aufgabe 2: Zählrohrcharakteristik: Inbetriebnahme Zählrohr - Einstellung der Einsatzspannung, dazu werden für verschiedene Zählrohr ~~Einsatzspannungen~~ ^{Spannungen} Zählraten gemessen und die Spannungskurve erstellt.

Einsatzspannung wurde parabolischem Signal als ~~500 V~~ gemessen.

$$U_E = (470 \pm 10) \text{ V}. \quad \Delta U_E = 10 \text{ V ist halbes Skalenfeld.}$$

Messungen ab $U_E \pm 50 \text{ V} = 520 \text{ V}$ für $t = 30 \text{ s}$ die ~~Zählrate~~ Ereignisse

$U [\text{V}]$	520 ± 10	550 ± 10	570 ± 10	600 ± 10	630 ± 10	690 ± 10	530	480
$N [\text{counts}]$	2639 2639	2747	2660	2737	2573	2622	2663	2283
$\Delta N = \sqrt{N}$								

$\Rightarrow$ Plateauspannung im Mittel $U_0 = 545 \text{ V} \pm 10 \text{ V}$

Aufgabe 3: Abstand d minimal, Messung für $t = 1 \text{ min} / 3 \text{ min}$ jeweils $U = U_0 / (U_0 + 100 \text{ V})$

	U_0	$U_0 + 100 \text{ V}$	
1 min 22327	7350 7277	7462 7353	$\Delta N = \sqrt{N}$
3 min 22327	229356 21834	222781 22327	Rechnung stimmt nicht

Aufgabe 4

2000 Messungen mit $t = 0.5 \text{ s}$

$$\mu = 61.455$$

comb.
0.5s

$$\sigma = 7.682$$

comb.
0.5sAufgabe 5 5000 Messungen mit $t = 0.7 \text{ s}$

$$\mu = 5.09$$

comb.
0.7s

$$\sigma = 2.26$$

comb.
0.7s

N. Rof

3. Auswertung

Formeln der statistischen Analyse¹

Varianzen: Für die statistische Auswertung von n Messwerten $\{x_i\}_{i=1}^n$ werden folgende Größen definiert:

$$\text{Arithmetisches Mittel} \quad \bar{x} = \text{Mean}(\{x_i\}_{i=1}^n) = \frac{1}{n} \sum_{i=1}^n x_i \quad (\text{S.1})$$

$$\text{Varianz} \quad \sigma^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2 \quad (\text{S.2})$$

$$\text{Standardabweichung} \quad \sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2} \quad (\text{S.3})$$

$$\text{Fehler des Mittelwerts} \quad \Delta \bar{x} = \frac{\sigma}{\sqrt{n-1}} \quad (\text{S.4})$$

Fehlerfortpflanzung: Der Fehler einer Funktion $f(x_1, \dots, x_N)$, die von den Variablen $\{x_i\}_{i=1}^N$ abhängt, wird wie folgt bestimmt:

$$\text{Gauß'sche Fehlerfortpfl.} \quad \Delta f = \sqrt{\sum_{i=1}^N \left(\frac{\partial f}{\partial x_i} \Delta x_i \right)^2} \quad (\text{S.5})$$

$$\text{relativer Fehler von } f = \prod_{i=1}^N x_i^{\alpha_i} \quad \frac{\Delta f}{|f|} = \sqrt{\sum_{i=1}^N \left(\frac{\alpha_i \Delta x_i}{x_i} \right)^2} \quad (\text{S.6})$$

Ergebnissignifikanz: Resultate eines Experiments bzw. Berechnung a_{exp} und die entsprechenden Literaturwerte a_{lit} werden folgend verglichen:

$$\begin{aligned} \text{Absolute Abweichung} \quad A &= |a_{\text{lit}} - a_{\text{exp}}| \\ \Delta A &= \sqrt{(\Delta a_{\text{lit}})^2 + (\Delta a_{\text{exp}})^2} \end{aligned} \quad (\text{S.7})$$

$$\text{Relative Abweichung} \quad R[\%] = \frac{A}{a_{\text{lit}}} \cdot 100 \quad (\text{S.8})$$

$$\text{Signifikanz} \quad S[\sigma] = \frac{A}{\Delta A} = \frac{|a_{\text{lit}} - a_{\text{exp}}|}{\sqrt{\Delta a_{\text{lit}}^2 + \Delta a_{\text{exp}}^2}} \quad (\text{S.9})$$

Fitgüte: Für einen Fit mit Funktionswerten $\{F_{a_1, \dots, a_m}(x_i)\}_{i=1}^n$ und Fitparametern $\{a_j\}_{j=1}^m$ an die experimentellen Messwerte $\{y_i\}_{i=1}^n$ mit Fehler $\{\Delta y_i\}_{i=1}^n$ kann die Güte des Fits mit diesen Größen analysiert werden:

$$\chi^2 \text{ Summe} \quad \chi^2 = \sum_{i=1}^n \left(\frac{F(x_i) - y_i}{\Delta y_i} \right)^2 \quad (\text{S.10})$$

$$\text{Freiheitsgrade} \quad f = n - m \quad (\text{S.11})$$

$$\text{normierte reduzierte } \chi^2 \text{ Summe} \quad \chi_{\text{red}}^2 = \frac{\chi^2}{f} \quad (\text{S.12})$$

$$\text{Fitwahrscheinlichkeit} \quad p = 1 - \text{chi2.cdf}(\chi^2, f) \quad (\text{S.13})$$

Hierbei wird das Modul `chi2` von `scipy.stats` benutzt.

3.1. Zählrohrspannung

Durch Auftragen der Messwerte von Ereignissen N (bei einer Totzeit von $t = 30$ s) bei Zählrohrspannung U aus Aufgabe 1 erhält man die Punkte in Abb. 3.1. Indem nun der erste und letzte Wert außer Acht gelassen werden, da diese nicht zum Plateaubereich der Spannungskurve gehören, kann ein linearer Fit im Plateau angelegt werden. Dieser ist erstmal irrelevant, die wichtige Essenz ist das Ablesen der einzustellenden Zählrohrspannung $U_0 = 545$ V als Mittelwert der Plateaugrenzen.

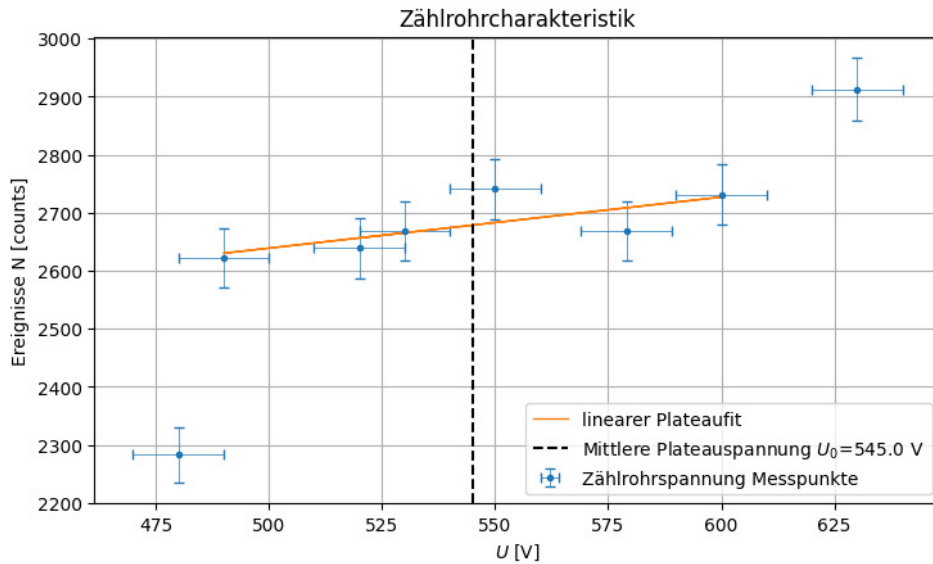


Abb. 3.1: Zählrohrcharakteristik - Spannungskurve

3.2. Plateaubereich des Zählers

Plateauanstieg: Es ist der Plateuanstieg zu bestimmen, der zwischen $N(U_0) - N(U_0 + 100 \text{ V})$ vorliegt, indem die Differenz der Zählraten $n = \frac{N}{t}$ bei erhöhtem U_0 betrachtet werden. Aufgrund der statistischen Verteilung (Poisson Verteilung) besitzen die Messgrößen der Ereignisse N einen Fehler von $\Delta N = \sqrt{N} \Rightarrow \Delta n = \frac{\sqrt{N}}{t}$.

$$D = n_{100} - n_0 = \frac{N_{100}}{t} - \frac{N_0}{t}$$

$$\Delta D = \sqrt{(\Delta n_{100})^2 + (\Delta n_0)^2} = \sqrt{\frac{(\Delta N_{100})^2 + (\Delta N_0)^2}{t^2}} = \sqrt{\frac{N_{100} + N_0}{t^2}} \quad (3.1)$$

Wäre das Plateau konstant, so sollte natürlich D_{60} null betragen. Die Abweichung von 0 kann nun mittels der Standardabweichung ΔD untersucht werden: $S[\sigma] = \frac{D}{\Delta D}$, man berechnet also, um wie viel $\sigma \equiv \Delta D$ D von null abweicht. Man erhält für zwei verschiedene Zeiten $t = (60, 180) \text{ s}$:

$$D_{60} = (2.4 \pm 2.1) \text{ s}^{-1} \Rightarrow S = 1.2 \quad D_{180} = (2.7 \pm 1.2) \text{ s}^{-1} \Rightarrow S = 2.3$$

Der prozentuale Anstieg wird folgend berechnet:

$$D_{\text{rel}} = \frac{D}{n_0} = \frac{Dt}{N_0} \quad \Delta D_{\text{rel}} = D_{\text{rel}} \sqrt{\frac{\Delta N_0^2}{N_0^2} + \frac{\Delta D^2}{D^2}} \quad (3.2)$$

Damit bekommt man noch $D_{60,\text{rel}} = (2.0 \pm 1.8) \% \quad D_{180,\text{rel}} = (2.2 \pm 1.0) \%$.

1% Genauigkeit des Anstiegs: Der Plateuanstieg soll durch lange Messdauer t auf $1 \% = \frac{D}{\Delta D}$ genau bestimmt werden: Dafür werden nun die gemessenen Raten n benutzt, deren Fehler kann wie in Gleichung (3.1) durch $\Delta n = \frac{\sqrt{N}}{t}$ ausgedrückt werden, und mit dem t im Nenner dann wiederum zu den Raten n umgewandelt werden ($(\frac{\sqrt{N}}{t})^2 = \frac{n}{t}$):

$$0.01 = \frac{\Delta D}{D} = \frac{\sqrt{(\Delta n_{100})^2 + (\Delta n_0)^2}}{n_{100} - n_0} = \frac{\sqrt{\frac{n_{100} + n_0}{t}}}{\sqrt{t}(n_{100} - n_0)} \leftrightarrow t = \frac{n_{100} + n_0}{0.01^2(n_{100} - n_0)^2} \quad (3.3)$$

$$\Delta t = 10000 \sqrt{\frac{\Delta_{n_0}^2 (n_0 + 3n_{100})^2 + \Delta_{n_{100}}^2 (3n_0 + n_{100})^2}{(n_0 - n_{100})^6}} \quad (3.4)$$

Damit berechnet man mithilfe der Raten $n = \frac{N}{180 \text{ s}}$ & $\Delta n = \frac{\sqrt{N}}{180 \text{ s}}$ der 3min Messung:

$$t = (300\,000 \pm 300\,000) \text{ s} = (5000 \pm 5000) \text{ h}$$

Die Fehler sind so hoch, da sich die Raten kaum unterschieden und damit $(n_{100} - n_0)^3$ im Nenner von Gleichung (3.4) sehr klein ist. Erst wenn die Raten über längere Zeit gemessen

werden, kompensiert der Zähler diesen Effekt, indem $\Delta n_{100/0}$ sehr klein wird, da $\Delta n \propto \frac{1}{t}$ mit der Messdauer t .

Variation der Zählrate: Meine Interpretation der Aufgabe: ΔD entspricht wie man oben gesehen hat, der Standardabweichung von D . Das heißt, wenn sich die Zählrate D um $(1, 2)\sigma$ bzw. $n\sigma$ ändert, also $D \pm n \cdot \Delta D$, dann betrachtet man den $n\sigma$ Sigma-Bereich, indem sich die Zählratendifferenz bewegen kann. Die prozentuale Variation $\text{Var}_{n\sigma}$ ist dann analog zu D_{rel} im Bezug auf n_0 zu ermitteln:

$$\frac{D \pm n \cdot \Delta D}{n_0} = \frac{D}{n_0} \pm \frac{n \cdot \Delta D}{n_0} = D_{\text{rel}} \pm \text{Var}_{n\sigma} \quad (3.5)$$

Diese Gleichung ergibt also ein Intervall, symmetrisch um $D_{\text{rel}} = \frac{D}{n_0}$, in welchem sich die Zählratendifferenz D bewegt - abhängig des Sigma-Bereiches. Die Aussage der Größe $\text{Var}_{n\sigma}$ ist also: Mit der Wahrscheinlichkeit von (68, 95) % liegt bei einer Wiederholungsmessung der Wert für $D'_{\text{rel}} \in (D_{\text{rel}} \pm \text{Var}_{1,2\sigma})$. Damit bekommt man für die zwei Zeiten:

$$\text{Var}_{(1,2)\sigma}^{60} = (1.75, 3.5) \% \quad \text{Var}_{(1,2)\sigma}^{180} = (0.98, 1.97) \%$$

3.3. Verifizierung der statistischen Natur radioaktiven Zerfalls

Durch Aufnehmen der Zerfallsereignisse N pro Torzeit $t = (0.5, 0.1)\text{s}$ und das 2000 bzw. 5000-Mal wurde ein Set an Ereigniszahlen generiert, die vom Messprogramm am Experimentiercomputer automatisch in ein Histogramm nach Ereigniscounts (wie häufig trat der Zerfall von N auf) sortiert wurden, sodass eine Häufigkeitsverteilung erstellt werden konnte.

Mittels der Scipy-Funktion `curvefit` kann eine Gauß- und Poissonverteilung an die Messdaten gefittet werden. Für die Gaußfunktion wurden als Parameter die Fläche A_g , der Mittelwert μ_g und die Standardabweichung σ_g als Parameter gewählt, für die Poissonverteilung die Fläche A_p und der Mittelwert μ_p . Außerdem wurden Datenpunkte mit Häufigkeiten < 10 nicht in den Fit einbezogen, da dieser dadurch verfälscht werden könnte. Außerdem wurden gemäß Abschnitt 3.(Formeln der statistischen Analyse¹) die χ^2 Summe, χ_{red}^2 und die Fitwahrscheinlichkeit p berechnet.

Statistik des radioaktiven Zerfalls: 2000 Messungen bei Torzeit 0.5s

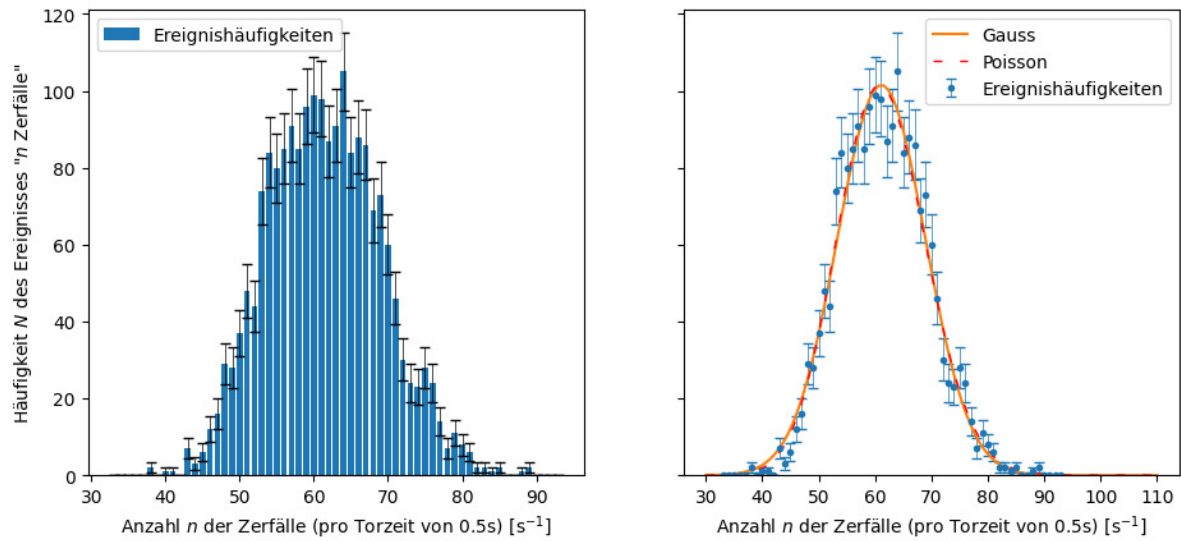


Abb. 3.2: Histogramm und Fit für 2000 Messungen

Statistik des radioaktiven Zerfalls: 5000 Messungen bei Torzeit 0.1s

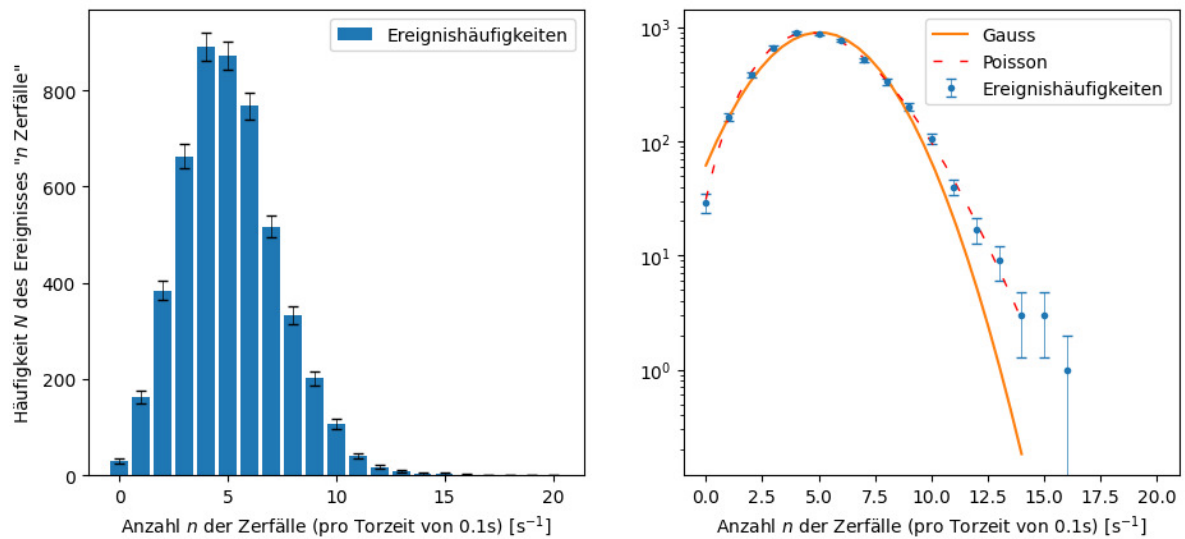


Abb. 3.3: Histogramm und Fit für 5000 Messungen, links nicht logarithmische Skala, um Asymetrie der Verteilung zu sehen

Die Fitergebnisse werden in dieser Tabelle zusammengefasst:

Tabelle 3.1: Fitergebnisse der statistischen Natur

Messanzahl	Parameter	Gaußfit	Poissonfit
2000	A	1990 ± 50	1990 ± 50
	$\mu[\frac{1}{0.5s}]$	61.12 ± 0.21	61.30 ± 0.21
	$\sigma[\frac{1}{0.5s}]$	7.82 ± 0.19	—
	χ^2	40.0	30.0
	χ^2_{red}	1.1	1.0
	p	40.0 %	52.0 %
5000	A	4900 ± 80	4990 ± 80
	$\mu[\frac{1}{0.1s}]$	5.04 ± 0.04	5.08 ± 0.04
	$\sigma[\frac{1}{0.1s}]$	2.174 ± 0.025	—
	χ^2	120.0	7.0
	χ^2_{red}	12.0	0.6
	p	0.0 %	85.0 %

4. Diskussion

4.1. Plateauanstieg

Der Plateauanstieg D für die Messzeiten $t = (60, 180)$ s zeigt bei längerer Messdauer geringeren Fehler, da dieser $\propto \frac{1}{t}$. Durch kleineren Fehler steigt die Signifikanz von 1.3σ bei 60 s auf 2.3σ bei 180 s. Der relative Anstieg D_{rel} ist bei beiden Messungen gleich hoch, dies entspricht der Erwartung, dass die Zählrohrcharakteristik unabhängig der Messzeit ist.

4.2. Messdauer für 1% Genauigkeit

Wie schon in der Auswertung beschrieben, der Fehler der 1%-Zeit ist sehr hoch aufgrund niedriger Differenz zwischen $n_{100} - n_0$. Diese Differenz kann zwar kompensiert werden, dafür bräuhete es aber deutlich längere Messdauern t , mit denen $n_{0/100}$ bestimmt wurden, um $\Delta n \propto \frac{1}{t}$ klein zu bekommen, die hier vorliegenden 3min reichen nicht aus. Im Prinzip steckt in dieser Rechnung aber auch keine weitere Erkenntnis als *für höhere Genauigkeiten, erhöhe die Messzeit* (gilt eben auch für $\frac{\Delta D}{D} \propto \frac{1}{\sqrt{t}}$).

4.3. Statistische Natur: Fits

2000 Messung: Am Histogramm sieht man gut die Symetrie der Verteilung aufgrund des hohen Mittelwerts. Dadurch kann die Poissonverteilung sehr gut durch die Gaußverteilung genähert werden. Die Poissonverteilung zeigt weiterhin bessere Werte: p beträgt fast genau 50 %, das ist der bestmögliche Wert für die Fitwahrscheinlichkeit. χ^2_{red} ist bei beiden Fits fast genau 1, was der optimale Wert ist.

5000-Messung: Wie man bei den Ergebnissen (Abb. 3.3, sowie der Tabelle 3.1) sieht, unterscheiden sich die Gauss- und Poisson Fits bei der 5000-Messung. Wie im Skript¹ beschrieben, liegt das an der Symetrie einer Gaußverteilung, die natürlich massiv im Widerspruch zur bei den Daten vorliegende Asymetrie (zu sehen am Histogramm) aufgrund kleinen Mittelwerts ($\mu < 30$) liegt. Aus diesem Grund sind die Fitparameter für die Gauss-Messung auch sehr schlecht ($\chi^2_{\text{red}} = 12 \gg 1$). Selbst beim Poissonfit liegt der reduzierte χ^2 Wert bei 0.6, also auch nicht optimal, aber weitaus besser als beim Gauß.

In beiden Messungen beträgt der Flächeninhalt nicht genau die Anzahl der Messungen, was daran liegt, dass die sehr selten aufgetretene Häufigkeiten/die Ränder der x-Achsen aus dem Fit herausgenommen wurden, aber natürlich auch im realen Experiment aufgetreten sind. Im Messprotokoll wurden die vom Messprogramm errechneten Werte für μ und σ aufgeschrieben, welche vor Ort direkt beim Aufnehmen der Messungen angezeigt wurden. Diese Werte sind in derselben Größenordnungen, wie in der Auswertung mit `curve_fit` ermittelten Werte.

4.4. Consistency Check

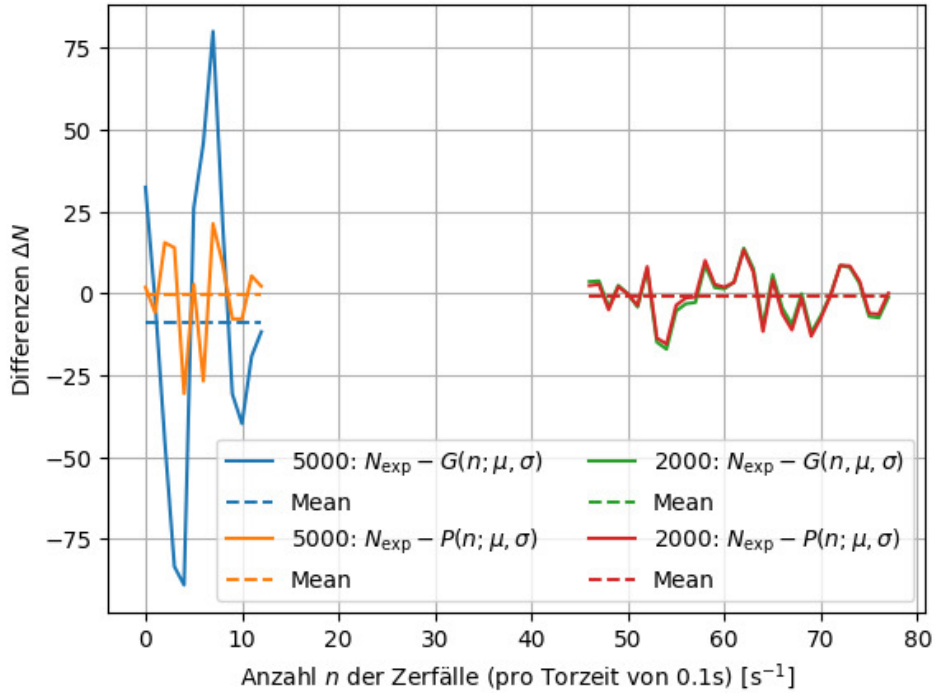


Abb. 4.1: Abweichungen der Mess- und Fitwerte

Rein aus Interesse habe ich die Abweichungen der Messwerte von den entsprechenden Fitwerten berechnet (was im Prinzip die χ^2 Summe macht) und über das gemessene Intervall geplottet, das ist die Größe $\Delta N = N_{\text{exp}} - G(n; \mu, \sigma)$ analog die Poissonverteilung $P(n; \mu)$ mit den jeweiligen Verteilungsmittelwerten μ bzw. Standardabweichungen σ . Die Größe N zählt die Häufigkeit der eingetretenen Ereignisse, dass in der Torzeit eine Anzahl n Teilchen zerfallen und detektiert worden sind.

Nevermind, das macht hier gar keinen Sinn

Wenn man nicht den Betrag (was eine evtl. χ^1 Summe wäre) analysiert, können sich negative und positive Differenzen aufheben bruh.

Warum habe ich damit jetzt meine Zeit gewastet uff. Um wenigstens dann das gemachte ein wenig nutzen zu können, habe ich die einzelnen Elemente der χ^2 Summe geplottet, also

$$f(n_i) := \chi_i^2 = \frac{N_{i,\text{exp}} - G_{\mu,\sigma}(n_i)}{\Delta N_{i,\text{exp}}} \Rightarrow \chi^2 = \int f(n) d\mu \quad (4.1)$$

Einzig schöne Beobachtung: Die χ^2 Summe entspricht also dem Integral über die im Diagramm gezeigte Kurve, nur mit dem Zählmaß $d\mu$ auf den natürlichen Zahlen. Man sieht also in Abb. 4.2, dass die Fits für die 2000 Messung fast annähernd identisch sind (χ_i^2 Werte fast gleich) und

beide Fits ziemlich gut sind, da das Integral sehr gering ist (zumindest im Gegensatz zum 5000er Gauss). Die Näherung $P \approx G$ für große μ wurde also bestätigt.

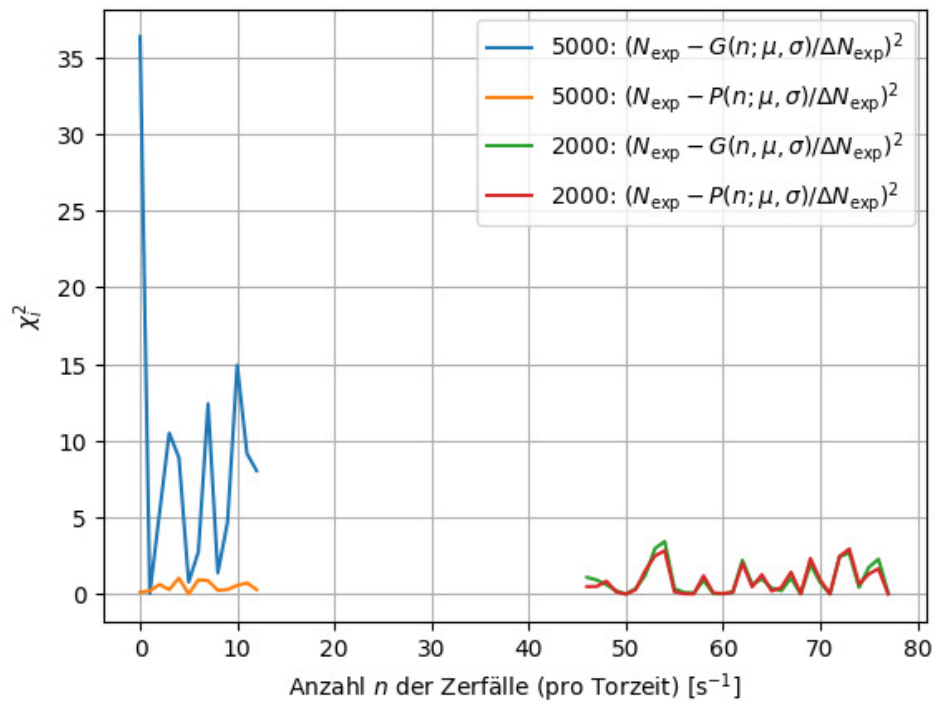


Abb. 4.2: χ^2 Summe grafisch dargestellt

4.5. Fazit

Radiaktivität ist sehr spannend. Die Gefahr nicht sehen zu können, lässt einen sehr schnell unvorsichtig werden. man muss sich gut konzentrieren, das habe ich auf jeden Fall gelernt.

Literaturquellen

¹Dr. J. Wagner, *Physikalisches Praktikum 2.2 für Studierende der Physik B.Sc.* [Online; Stand 04/2026] (Universität Heidelberg, Apr. 2026), https://git.physi.uni-heidelberg.de/schwierz/Versuchsanleitungen/src/branch/main/PAP2_2.pdf (besucht am 14.04.2026) (siehe S. 3, 5–8, 11, 14, 17).

Abbildungsquellen

²City council of Pripjat, *Pripyat evacuation announcement*, [Online; Stand 05/2026], <https://www.youtube.com/watch?v=P12i0G29-PI> (siehe S. 1).

³Wikipedia, *ISO 7010*, [Online; Stand 05/2026], https://de.wikipedia.org/wiki/ISO_7010 (siehe S. 1).

A. Code-Anhang

Bei den Funktionen habe ich teilweise Dinge von verschiedenen Altprotokollen zusammengesucht, und auf meine Bedürfnisse umgeschrieben. Die konkreten Berechnung sind dann eigenständig.

July 11, 2026

```
[235]: # Numpy für bessere Berechnungen
import numpy as np
from numpy import exp, sqrt, log, pi
from uncertainties import unumpy as unp

# Weiteres für bessere Rechnungen
import pylab as py

from decimal import Decimal, ROUND_CEILING, ROUND_HALF_UP, getcontext #L
↳ Besonders für sig. Runden
import math

# Berechnungen und Plotting
from scipy import odr
import scipy.optimize
import scipy.constants as c
from scipy.stats import norm
from scipy.optimize import curve_fit
from scipy.stats import chi2
from scipy.stats import poisson
from scipy.integrate import quad
from scipy.signal import find_peaks
from scipy.signal import argrelextrema, argrelemin, argrelemax
from scipy.special import factorial
import scipy.integrate as integrate
from scipy.special import gamma

%matplotlib widget
import matplotlib.pyplot as plt
from matplotlib import colormaps
import matplotlib.mlab as mlab
import matplotlib.transforms as transforms

# Zum Auslesen von Dateien und ähnlichem
import os
import os.path
from pathlib import Path
```

```

import pandas as pd # Auch wichtig für den Latex Export
from pytexit import py2tex
import csv
import re
from typing import List

# Besseres Funktionen handling
import sympy as sp
from sympy import separatevars

# Display und Output
from IPython.display import display, Math, Latex, HTML

import textwrap

# Grafiken in EU-Größen ausgeben
def figsize_cm(width_cm, height_cm):
    return (2*width_cm / 2.54, 2*height_cm / 2.54)
def comma_to_float(valstr):
    return float(valstr.replace(',', '.'))

g=9.80984
Delta_g=0.00002

```

```

[236]: currentversuch = "251"
base = Path.cwd()
Messwerte_path = Path(base.parent.parent.parent/"Messwerte"/currentversuch)
↳ # initializing paths
Ausgabe_path = Path(base/"Diagramme")
print(Messwerte_path)
print(Ausgabe_path)

```

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2\Messwerte\251

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2\Auswertungen2.2\Code\251\Diagramme

Delete UTF8 False encoding

```

[237]: # def deletevocals(text):
#     text=textwrap.dedent(text)
#     patterna = r'[][a]'
#     text=re.sub(patterna,"ä",text)
#     patternu=r'[][u]'
#     text=re.sub(patternu,"ü",text)
#     patterno=r'[][o]'
#     text=re.sub(patterno,"ö",text)
#     return text

```

```

# Regexpwissen: [0-9], [A-Z] findet egal welchen Zahl/Buchstaben; [^0-9] findet
↳ erstes Zeichen, das keine Zahl ist;

def deletevowels(text):
    # Einrückungen entfernen (gemeinsame Tabs des gesamten Textes)
    text = textwrap.dedent(text)
    # entfernt Bindestriche: - findet Bindestrich, \n Zeilenumbruch direkt
↳ dahinter, \s Whitespaces *beliebig viele
    text = re.sub(r'[-\n\s*]', '', text)
    # entfernt Zeilenumbrüche
    text = text.replace("\n", " ")

    # Eine Funktion, die vom re.sub aufgerufen wird, wenn ein Treffer erzielt
↳ wird
    def convert(match):
        # match.group(1) ist der Buchstabe nach dem Pünktchen (z.B. 'a' oder
↳ 'A')
        buchstabe = match.group(1)
        # Dictionary für die echten Umlaute
        umlaute = {'a': 'ä', 'u': 'ü', 'o': 'ö', 'A': 'Ä', 'U': 'Ü', 'O': 'Ö'}
        # Gib den Umlaut zurück. Falls das Zeichen nicht im Dict ist, lass es
↳ wie es ist
        return umlaute.get(buchstabe, buchstabe)

    # Das Pattern sucht nach " gefolgt von einem beliebigen Buchstaben aus der
↳ Auswahl, [X] findet genau ein Zeichen aus Menge X
    return re.sub(r'"([auoAUO])"', convert, text)

```

```

[238]: print(deletevowels("""
Die qualitativen Beobachtungen des Rauschens entsprachen im Grunde der
↳ theoretischen Erwartung:
Einerseits besagt die Nyquist-Beziehung gerade, dass die Effektivspannungen
↳ (das beobachtete Rau-
schen) mit erhöhten Widerstānden steige
"""))

```

Die qualitativen Beobachtungen des Rauschens entsprachen im Grunde der theoretischen Erwartung: Einerseits besagt die Nyquist-Beziehung gerade, dass die Effektivspannungen (das beobachtete Rauschen) mit erhöhten Widerständen steige

Runden signifikanter Stellen

```

[239]: def round_to_sigs(val, errVal=None, einheiten=None):
    """
    Funktion zur Rundung eines Fehlers und die Anpassung des Messwertes daran.
↳ Diese Funktion wurde etwas umständlicher

```

geschrieben, da python mit Floats und Runden schnell in Probleme rennt.
↳ Daher musste hier mit dezimal gearbeitet werden.

Zudem sollten besonders kleine und große Messwerte in der
↳ Dezimalschreibweise geschrieben werden, damit diese auch
für Protokolle geeignet sind.

Parameter

****val**** : float
Messwert

****errVal**** : float
Ungenauigkeit des Messwertes

Return

****value_rounded**** : str
Gerundeter Messwert

****error_rounded**** : str
Gerundete Ungenauigkeit des Messwertes

****res**** : str
"Messwert \pm Fehler"
"""

```
einheiten_str = einheiten if einheiten is not None else ""  
if errVal is not None:  
    # Daten zu Dezimal wechseln, da Floats probleme machen  
    val = Decimal(str(val))  
    errVal = Decimal(str(errVal))
```

```
    exp = int(math.floor(math.log10(float(errVal))))  
↳ signifikante Stellenposition wird anhand des errVal bestimmt
```

```
    if round(float(errVal / (Decimal(10) ** exp))) < 3: # wenn   
↳ 1,2,3 erste signifikante Stelle, dann kommt eine zweite
```

```
↳ Nachkommastellenposition hinzu  
        exp -= 1
```

```
    scale = Decimal(10) ** exp
```

```
    val_round = (val / scale).quantize(Decimal('1'),  
↳ rounding=ROUND_HALF_UP) * scale # mit scale wird das Komma so  
↳ verschoben, dass alle signifikanten Stellen vor dem Komma stehen, und dann  
↳ alle Nachkommastellen weggerundet werden
```

```
    err_round = (errVal / scale).quantize(Decimal('1'),  
↳ rounding=ROUND_CEILING) * scale
```

```

        num = f"\\num{{{val_round}}({err_round})}"
        qty = f"\\qty{{{val_round}}({err_round})}{{{einheiten_str}}}" if
↪einheiten else num

        return float(val_round), float(err_round), num, qty
    else:
        val = Decimal(str(val))
        exp = int(math.floor(math.log10(float(val))))
        if round(float(val / (Decimal(10) ** exp))) < 3:           # wenn 1,2,3
↪erste signifikante Stelle, dann kommt eine zweite Nachkommastellenposition
↪hinzu
            exp -= 1
            scale = Decimal(10) ** exp
            val_round = (val / scale).quantize(Decimal('1'),
↪rounding=ROUND_HALF_UP) * scale
            num = f"\\num{{{val_round}}}"
            qty = f"\\qty{{{val_round}}}{{{einheiten_str}}}" if einheiten else num
        return float(val_round), None, num, qty

v_round = np.vectorize(round_to_sigs, otypes=[float, float, object, object],
↪excluded=['einheiten'])

    # wissenschaftliche Notation erstmal vernachlässigen (da häufig Einheiten
↪gewollt sind, die 10^/e Präfixe beinhalten)

```

Gauss'sche Fehlerfortpflanzung

```

[240]: def gff(function, errPronePar, latex=True):
        """
        Kann die Fehlerformel einer gegebenen Gleichung bestimmen.

        Parameters
        -----
        **func** : sympy function
            Funktion dessen Fehler bestimmt werden soll.

        **errPronePar** : Array
            Liste (Array) aller fehlerbehafteten Größen der Gleichung con sp.Symbols
            Diese Werte werden als x_sym, y_sym, z_sym etc. bezeichnet und sind
↪ungleich den Werten für x, y, z.
            Für die Werte wird daher die Bezeichnung x_val, y_val, z_val etc.
↪genutzt und für deren Fehler err_x, err_y, err_z etc.

        Return
        -----
        **absolut_err** : sympy function

```

```

        Gibt die Fehlergleichung des absoluten Fehlers wieder.

    **relativ_err** : sympy function
        Gibt die Fehlergleichung des relativen Fehlers wieder.

    **errProneParamters** : array
        Liste aller Fehlerbehafteten Größen
    """

    error = 0
    errProneParamaters = []

    function = sp.sympify(function)
    variables = errPronePar.split(",")
    variables = sp.symbols(variables)

    for variable in variables:
        Delta = sp.Symbol(f"Delta_{variable}")
        partial = sp.diff(function, variable) # Die Funktion
        ↪wird nach der fehlerbehafteten Variable abgeleitet
        term=sp.UnevaluatedExpr(partial*Delta)**2
        error = error + ((term)) # Fehler werden
        ↪quadratisch aufsummiert
        errProneParamaters.append((variable,Delta))

    absolute_err=sp.simplify(sp.sqrt(error),rational = True)
    relative_err=sp.simplify(sp.sqrt(error/function**2),rational = True)

    if latex:
        #Prints out the Latex Code for the Functions
        print("Funktion:")
        display(Math(sp.latex(function,long_frac_ratio=2)))
        print(sp.latex(function))
        print("Absoluter Fehler:")
        display(Math(sp.latex(absolute_err,long_frac_ratio=2)))
        print(sp.latex(absolute_err))
        print("Relativer Fehler:")
        display(Math(sp.latex(relative_err,long_frac_ratio=2)))
        print(sp.latex(relative_err))
    return function,absolute_err, relative_err, errProneParamaters

```

Sigma-Abweichungen

```

[241]: def sigma_abweichung(p1, err_p1,p2,err_p2):
        """
        Funktion zum berechnen der Sigma-Abweichugn von zwei Messwerten, oder einem
        ↪Messwert und einem Literaturwert.

```

```

"""
if err_p1 == 0 and err_p2 == 0:
    raise ValueError("Für die Sigma-Abweichung muss mindestens ein Wert_
↪fehlerbehaftet sein!")
else:
    abweichung=Decimal(str(abs(p1 - p2)/(np.sqrt(err_p1 ** 2+err_p2 ** 2))))

if abweichung == 0:
    return 0.0, "\\num{0}"

exp = int(math.floor(math.log10(float(abweichung))))
if round(float(abweichung / (Decimal(10) ** exp))) < 3:           # wenn_
↪1,2,3 erste signifikante Stelle, dann kommt eine zweite_
↪Nachkommastellenposition hinzu
    exp -= 1
scale = Decimal(10) ** exp
abweichung_round = (abweichung / scale).quantize(Decimal('1'),_
↪rounding=ROUND_CEILING) * scale
res = f"\\num{{{abweichung_round}}}"

return float(abweichung_round),res

```

```

[242]: # #Größenvergleich in latex
# def size_comp_str(name1,name2,val1,val2):
#     if val1<val2:
#         return name1+"<"+name2
#     elif val1>val2:
#         return name1+">"+name2
#     else:
#         return name1+"="+name2

#Erstellung von Vergleichstabellen in Latex
def compare_table_array(nam1, nam2, einheiten, val1_array, err1_array,_
↪val2_array, err2_array):
    # 1. Tabellenkopf (Hier werden die Strings einmalig eingesetzt)
    output_header = textwrap.dedent(f"""
        \\begin{{table}}[htb]
        \\centering
        \\caption{{Vergleich von ${nam1}$ und ${nam2}$}}
        \\begin{{tabularx}}{{0.8\\textwidth}}{{ZZZZZ}}
        \\toprule
        Messreihe & {nam1}[\\unit{{{einheiten}}}] &_
↪{nam2}[\\unit{{{einheiten}}}] & \\text{{abs.Abw.}}[\\unit{{{einheiten}}}] &_
↪\\text{{proz.Abw.}}[\\unit{{{percent}}}] & S[\\sigma] \\midrule
        """).strip() # strip macht whitespaces am anfang und ende des strings weg

    table_rows = []

```

```

# 2. Der Body (Schleife läuft über die Werte-Arrays)
for val1, err1, val2, err2 in zip(val1_array, err1_array, val2_array,
err2_array):

    # Deine Berechnungen (bleiben exakt gleich)
    VAL1 = round_to_sigs(val1, err1, r'')[2]
    VAL2 = round_to_sigs(val2, err2, r'')[2]

    abs_val = np.abs(val1 - val2)
    abs_delta = np.sqrt(err1**2 + err2**2)

    if abs_delta != 0:
        SIGMA = sigma_abweichung(val1, err1, val2, err2)[1]
    else:
        SIGMA = 0.0

    ABS = round_to_sigs(abs_val, abs_delta, r'')[2]

    rel = abs_val / np.abs(val1) * 100 if val1 != 0 else 0
    rel_delta = rel * np.sqrt((abs_delta / abs_val)**2 + (err1 / val1)**2)
if abs_val != 0 and val1 != 0 else 0
    REL = round_to_sigs(rel, rel_delta, r'')[2]

    # Eine saubere Zeile nur mit den Werten für LaTeX
    row = f"          & {VAL1} & {VAL2} & {ABS} & {REL} & {SIGMA} \\\\"
    table_rows.append(row)

body_str = "\\n".join(table_rows)

# 3. Tabellenfuß
output_footer = textwrap.dedent(f"""
    \\bottomrule
    \\end{{tabularx}}
    \\label{{table:Vergleich_{{nam1}}}}
    \\end{{table}}
""").strip()

# Alles zusammensetzen
final_output = f"{output_header}\\n{body_str}\\n{output_footer}"

print(final_output)

```

```

[243]: # #Erstellung von Vergleichstabellen in Latex
# def compare_table_withliterature(nam1,nam2,einheiten,val1,err1,val2):
#     VAL1=round_to_sigs(val1,err1,r'')[2]
#     abs=np.abs(val1-val2)

```

```

#     abs_delta=np.sqrt(err1**2)
#     if abs_delta!=0:
#         SIGMA=sigma_abweichung(val1,err1,val2,0)[1]
#     else:
#         SIGMA=0.0
#     ABS=round_to_sigs(abs,abs_delta,r')[2]
#     rel=abs/np.abs(val2)*100
#     rel_delta=rel*np.sqrt((abs_delta/abs)**2+(err1/val1)**2) if abs != 0 else 0
#     REL=round_to_sigs(rel,rel_delta,r')[2]
#     output = textwrap.dedent(f"""
#         \begin{{table}}[htb]
#         \centering
#         \caption{{}}
#         \begin{{tabularx}}{{0.8\textwidth}}{{ZZZZZ}}
#         \toprule
#         \hline
#         \rightarrow{{nam1}}[\\unit{{{{einheiten}}}}]&{{nam2}}[\\unit{{{{einheiten}}}}]&\\text{{abs.Abw.
#         \rightarrow}}[\\unit{{{{einheiten}}}}]&\\text{{proz.Abw.
#         \rightarrow}}[\\unit{{{{percent}}}}]&S[\\sigma]\\\\\\midrule
#         {VAL1}&\\num{{{{val2}}}}&{{ABS}}&{{REL}}&{{SIGMA}}\\\\
#         \bottomrule
#         \end{{tabularx}}
#         \label{{table:Vergleich_{{nam1}}}}
#         \end{{table}}
#     """)
#     print(output)

# #Größenvergleich in latex
# def size_comp_str(name1,name2,val1,val2):
#     if val1<val2:
#         return name1+"<"+name2
#     elif val1>val2:
#         return name1+">"+name2
#     else:
#         return name1+"="+name2

#Erstellung von Vergleichstabellen in Latex
import textwrap
def compare_table(nam1,nam2,einheiten,val1,err1,val2,err2):
    VAL1=round_to_sigs(val1,err1,r')[2]
    if err2!=0:
        VAL2=round_to_sigs(val2,err2,r')[2]
    else:
        VAL2=val2

```

```

abs=np.abs(val1-val2)
abs_delta=np.sqrt(err1**2+err2**2)
if abs_delta!=0:
    SIGMA=sigma_abweichung(val1,err1,val2,err2)[1]
else:
    SIGMA=0.0
ABS=round_to_sigs(abs,abs_delta,r')[2]
rel=abs/np.abs(val1)*100
rel_delta=rel*np.sqrt((abs_delta/abs)**2+(err1/val1)**2) if abs != 0 else 0
REL=round_to_sigs(rel,rel_delta,r')[2]
output = textwrap.dedent(f"""
    \\begin{{table}}[htb]
    \\centering
    \\caption{{}}
    \\begin{{tabularx}}{{\\textwidth}}{{ZZZx}}
    \\toprule
    \\
    ↪{nam1}[\\unit{{{{einheiten}}}}]&{nam2}[\\unit{{{{einheiten}}}}]&\\text{{abs.Abw.
    ↪}}[\\unit{{{{einheiten}}}}]&\\text{{proz.Abw.
    ↪}}[\\unit{{{{percent}}}}]&S[\\sigma]\\\\\\midrule
    {VAL1}&{VAL2}&{ABS}&{REL}&{SIGMA}\\\\
    \\bottomrule
    \\end{{tabularx}}
    \\label{{table:Vergleich_{nam1}}}
    \\end{{table}}
""")
print(output)

```

Zählrohrcharakteristik

```

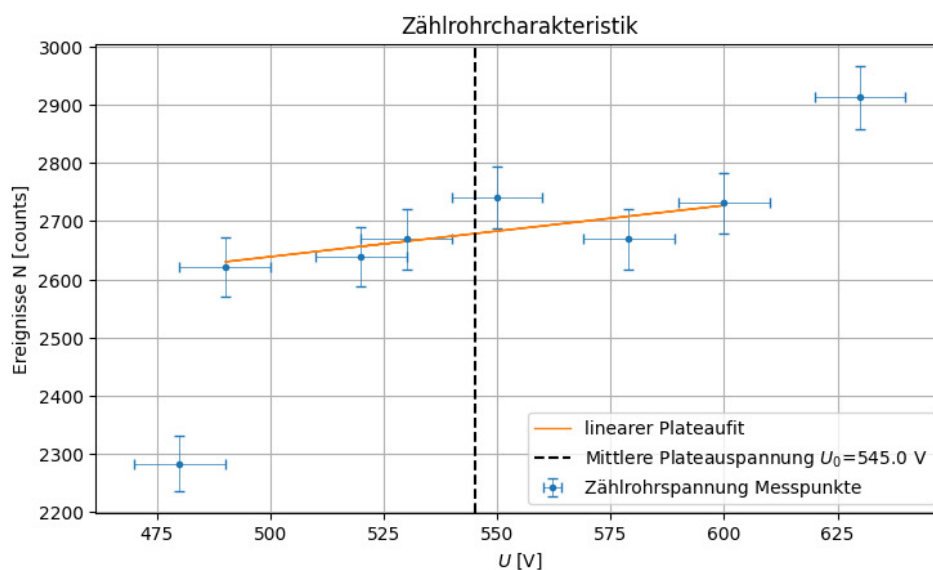
[244]: plt.close('all')
plt.figure(figsize=(figsize_cm(10.8,6)))
plt.title(r'Zählrohrcharakteristik')
plt.xlabel(r'$U$ [V]')
plt.ylabel(r'Ereignisse N [counts]')

U=np.array([520,550,579,600,630,490,530,480])
Delta_U=10
N=np.array([2639,2741,2669,2731,2913,2622,2669,2283])
Delta_N=np.sqrt(N)

plt.errorbar(U,N,xerr=Delta_U,yerr=Delta_N,capsize=3,elinewidth=0.
    ↪5,label=rf'Zählrohrspannung Messpunkte',fmt='.')
def lin(x,a,b):
    return a*x+b
U=np.delete(U,[-1,-4])
popt, pcov=curve_fit(lin, U,np.delete(N,[-1,-4]))

```

```
plt.plot(U, lin(U, *popt), label='linearer Plateaufit', linewidth=1, linestyle='-')
middle_U = np.min(U) + (np.max(U) - np.min(U)) / 2
plt.axvline(middle_U, linestyle='--', color='black', label=rf'Mittlere_
↳Plateauspannung $U_0$={middle_U} V')
plt.legend()
plt.grid()
# plt.tight_layout()
plt.savefig(Ausgabe_path/"Zählrohrcharakteristik.
↳png", format="png", bbox_inches='tight')
plt.show()
```



Plateauanstieg

```
[245]: N_0_1=7211
Delta_N_0_1=sqrt(N_0_1)
N_0_3=21834
Delta_N_0_3=sqrt(N_0_3)
N_100_1=7353
Delta_N_100_1=sqrt(N_100_1)
N_100_3=22327
Delta_N_100_3=sqrt(N_100_3)

D_1=N_100_1/60-N_0_1/60
Delta_D_1= np.sqrt((Delta_N_0_1**2 + Delta_N_100_1**2)/60**2)
D_1,Delta_D_1,_,latexD_1=round_to_sigs(D_1,Delta_D_1,r'\per\s')
print(latexD_1)
print(round_to_sigs(D_1/Delta_D_1,0.01,r'')[0])
```

```

print(sigma_abweichung(N_0_1,Delta_N_0_1,N_100_1,Delta_N_100_1)[1])
rel=D_1/N_0_1*60*100
Delta_rel=rel*np.sqrt(Delta_N_0_1**2/N_0_1**2 + Delta_D_1**2/D_1**2)
rel,Delta_rel,_,latexrel=round_to_sigs(rel,Delta_rel,r'\percent')
print(latexrel)

D_3=N_100_3/180-N_0_3/180
Delta_D_3=np.sqrt((Delta_N_0_3**2 + Delta_N_100_3**2)/180**2)
D_3,Delta_D_3,_,latexD_3=round_to_sigs(D_3,Delta_D_3,r'\per\s')
print(latexD_3)
print(round_to_sigs(D_3/Delta_D_3,0.01,r')[0])
print(sigma_abweichung(N_0_3,Delta_N_0_3,N_100_3,Delta_N_100_3)[1])
rel=D_3/N_0_3*180*100
Delta_rel=rel*np.sqrt(Delta_N_0_3**2/N_0_3**2 + Delta_D_3**2/D_3**2)
rel,Delta_rel,_,latexrel=round_to_sigs(rel,Delta_rel,r'\percent')
print(latexrel)

N_0_3=21834
N_100_3=22327
n_0=N_0_3/180
Delta_n_0=sqrt(N_0_3)/180
n_100=N_100_3/180
Delta_n_100=sqrt(N_100_3)/180
t=(n_100+n_0)/(0.01**2*(n_100-n_0)**2)
Delta_t= 10000*np.sqrt((Delta_n_0**2*(n_0 + 3*n_100)**2 + Delta_n_100**2*(3*n_0_
↵+ n_100)**2)/(n_0 - n_100)**6)
t,Delta_t,_,latext=round_to_sigs(t,Delta_t,r'\s')
print(latext)
print(300000/60)

variation_68 = (Delta_D_1 / (N_0_1/60)) * 100
variation_95 = (2 * Delta_D_1 / (N_0_1/60)) * 100
print(variation_68)
print(variation_95)
variation_68 = (Delta_D_3 / (N_0_3/180)) * 100
variation_95 = (2*Delta_D_3 / (N_0_3/180)) * 100
print(variation_68)
print(variation_95)

```

```

\qty{2.4(2.1)}{\per\s}
1.143
\num{1.2}
\qty{2.0(1.8)}{\percent}
\qty{2.7(1.2)}{\per\s}
2.25
\num{2.4}
\qty{2.2(1.0)}{\percent}
\qty{300000(300000)}{\s}

```

```

5000.0
1.7473304673415617
3.4946609346831234
0.9892827699917559
1.9785655399835118

```

```

[246]: # Deine Raten und Fehler aus der vorherigen Rechnung
n_0 = 21834 / 180
Delta_n_0 = np.sqrt(21834) / 180

n_100 = 22327 / 180
Delta_n_100 = np.sqrt(22327) / 180

# 1. Absolute Differenz (Effekt der Spannungserhöhung)
D = n_100 - n_0

# 2. Absoluter Fehler der Differenz
Delta_D = np.sqrt(Delta_n_0**2 + Delta_n_100**2)
print(D,Delta_D)
# 3. Prozentuale Variation für 68% (1 Sigma)
variation_68 = (Delta_D / D) * 100

# 4. Prozentuale Variation für 95% (2 Sigma)
variation_95 = (2 * Delta_D / D) * 100

print(f"Variation bei 68% Vertrauensniveau: {variation_68:.1f} %")
print(f"Variation bei 95% Vertrauensniveau: {variation_95:.1f} %")

```

```

2.73888888888888943 1.1674732661438092
Variation bei 68% Vertrauensniveau: 42.6 %
Variation bei 95% Vertrauensniveau: 85.3 %

```

```

[247]: n05,N = np.loadtxt(Messwerte_path/"data_aufgabe2.txt", delimiter="," ,
↳ skiprows=4, unpack=True) # Zerfälle pro 0.5s, Anzahl der Ereignisse, bei
↳ denen n5 aufgetreten sind
fehlerN=np.sqrt(N)
fehlerN2000=fehlerN[13:-16]
plt.close('all')
fig, (ax1,ax2) = plt.subplots(1, 2, figsize=(figsize_cm(14,6)),sharey=True)
fig.suptitle('Statistik des radioaktiven Zerfalls: 2000 Messungen bei Torzeit 0.
↳ 5s')

ax1.bar(n05,N,yerr=fehlerN,error_kw={'capsize': 3,'elinewidth': 0.
↳ 5},label='Ereignishäufigkeiten')
ax1.set_title('')
ax1.set_xlabel(r'Anzahl $n$ der Zerfälle (pro Torzeit von 0.5s) [s$^{-1}$]')
ax1.set_ylabel(r'Häufigkeit $N$ des Ereignisses "$n$ Zerfälle")

```

```

ax2.errorbar(n05, N, fehlerN,fmt=".",capsize=3,elinewidth=0.
    ↪5,label='Ereignishäufigkeiten')
ax2.set_xlabel(r'Anzahl $n$ der Zerfälle (pro Torzeit von 0.5s) [s$^{-1}$]')
def gaussian(x, A, mu, sig): #A: Fläche der Gaussfunktion
    return A/(sqrt(2*pi)*sig)*exp(-(x-mu)**2/2/sig**2)
popt, pcov=curve_fit(gaussian,n05[13:-16], N[13:
    ↪-16],p0=[2000,75,8],sigma=fehlerN[13:-16],absolute_sigma=True)
def poisson(x, A_p, mu_p):
    return A_p*exp(-mu_p)*mu_p**x/gamma(x+1)
popt_p, pcov_p = curve_fit(poisson, n05[13:-16], N[13:-16], p0=[2000,
    ↪75],sigma=fehlerN[13:-16], absolute_sigma=True)
x=np.linspace(30,110, 100)
ax2.plot(x, gaussian(x,*popt), label='Gauss')
ax2.plot(x, poisson(x,*popt_p), label='Poisson',color='red',linestyle=(0, (5,
    ↪10)),linewidth=1)
diff_2000=gaussian(x,*popt)-poisson(x,*popt_p)
x_diff_2000=np.arange(int(n05[13]),int(n05[-16]),1)
diff_2000_g=-N[13:-16]+gaussian(x_diff_2000,*popt)
diff_2000_p=-N[13:-16]+poisson(x_diff_2000,*popt_p)

ax1.legend()
# ax1.grid()
ax2.legend()
# ax2.grid()

plt.savefig(Ausgabe_path/'Histogramm0.5.png',format="png",bbox_inches='tight')
plt.tight_layout()
plt.show()

print("Gaussfit:")
print("A="+round_to_sigs(popt[0],np.sqrt(pcov[0][0]),r'')[3])
print("mu="+round_to_sigs(popt[1],np.sqrt(pcov[1][1]),r'\per{s'}[3])
print("sigma="+round_to_sigs(popt[2],np.sqrt(pcov[2][2]),r'\per{s'}[3])
print("Poissonfit:")
print(r"mu_p="+round_to_sigs(popt_p[1],np.sqrt(pcov_p[1][1]),r'\per{s'}[3])
print(r"A_p="+round_to_sigs(popt_p[0],np.sqrt(pcov_p[0][0]),r'')[3])

#Gauss:
chi2_g=np.sum((gaussian(n05[13:-16],*popt)-N[13:-16])**2/fehlerN[13:-16]**2)
dof_g=len(n05[13:-16])-3 #dof:degrees of freedom, Freiheitsgrad
chi2_red_g=chi2_g/dof_g
print("chi2_g=", round_to_sigs(chi2_g)[0])
print("chi2_red_g=",round_to_sigs(chi2_red_g)[0])
#Poisson:
chi2_p=np.sum((poisson(n05[13:-16],*popt_p)
-N[13:-16])**2/fehlerN[13:-16]**2)
dof_p=len(n05[13:-16])-2 #poisson hat nur 2 Parameter

```

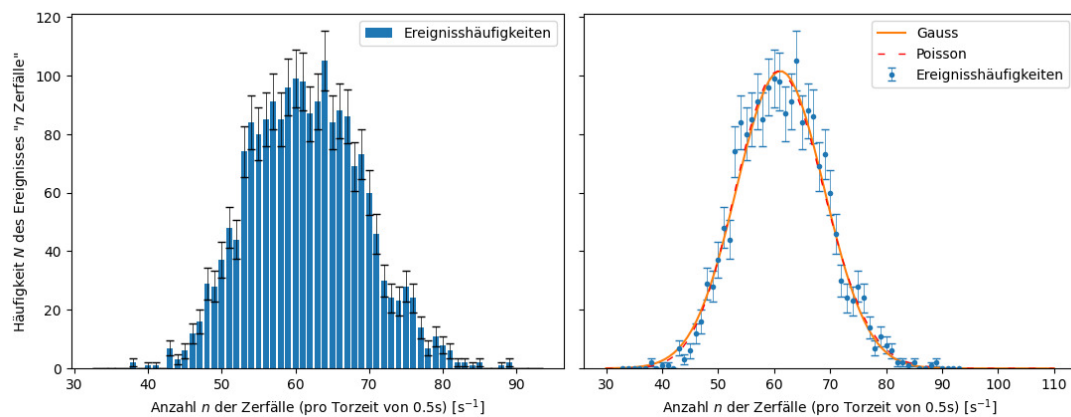
```

chi2_red_p=chi2_p/dof_p
print("chi2_p=", round_to_sigs(chi2_p)[0])
print("chi2_red_p=",round_to_sigs(chi2_red_p)[0])

from scipy.stats import chi2
#Gauss:
prob_g=round(1-chi2.cdf(chi2_g,dof_g),2)*100
#Poisson:
prob_p=round(1-chi2.cdf(chi2_p,dof_p),2)*100
print("Wahrscheinlichkeit Gauss=", prob_g,"%")
print("Wahrscheinlichkeit Poisson=", prob_p,"%")

```

Statistik des radioaktiven Zerfalls: 2000 Messungen bei Torzeit 0.5s



```

Gaussfit:
A=\num{1990(50)}
mu=\qty{61.12(0.21)}{\per\s}
sigma=\qty{7.82(0.19)}{\per\s}
Poissonfit:
mu_p=\qty{61.30(0.21)}{\per\s}
A_p=\num{1990(50)}
chi2_g= 30.0
chi2_red_g= 1.0
chi2_p= 30.0
chi2_red_p= 1.0
Wahrscheinlichkeit Gauss= 40.0 %
Wahrscheinlichkeit Poisson= 52.0 %

```

```

[248]: n05,N = np.loadtxt(Messwerte_path/"data_aufgabe3.txt", delimiter=",",
    ↪skiprows=4, unpack=True) # Zerfälle pro 0.5s, Anzahl der Ereignisse, bei
    ↪denen n5 aufgetreten sind
fehlerN=np.sqrt(N)
fehlerN5000=fehlerN[:-8]

```

```

plt.close('all')
fig, (ax1,ax2) = plt.subplots(1, 2, figsize=(figsize_cm(14,6)))
fig.suptitle('Statistik des radioaktiven Zerfalls: 5000 Messungen bei Torzeit 0.
↳1s')

ax1.bar(n05,N,yerr=fehlerN,error_kw={'capsize': 3,'elinewidth': 0.
↳5},label='Ereignishäufigkeiten')
ax1.set_title('')
ax1.set_xlabel(r'Anzahl $n$ der Zerfälle (pro Torzeit von 0.1s) [s$^{-1}$]')
ax1.set_ylabel(r'Häufigkeit $N$ des Ereignisses "$n$ Zerfälle")

ax2.errorbar(n05, N, fehlerN,fmt=".",capsize=3,elinewidth=0.
↳5,label='Ereignishäufigkeiten')
ax2.set_xlabel(r'Anzahl $n$ der Zerfälle (pro Torzeit von 0.1s) [s$^{-1}$]')
def gaussian(x, A, mu, sig): #A: Fläche der Gaussfunktion
    return A/(sqrt(2*pi)*sig)*exp(-(x-mu)**2/2/sig**2)
popt, pcov=curve_fit(gaussian,n05[:-8], N[:-8],p0=[5000,5,2],sigma=fehlerN[:
↳-8],absolute_sigma=True)
def poisson(x, A_p, mu_p):
    return A_p*exp(-mu_p)*mu_p**x/gamma(x+1)
popt_p, pcov_p = curve_fit(poisson, n05[:-8], N[:-8], p0=[5000,↳
↳5],sigma=fehlerN[:-8], absolute_sigma=True)
x=np.linspace(0,14, 30)
ax2.plot(x, gaussian(x,*popt), label='Gauss')
ax2.plot(x, poisson(x,*popt_p), label='Poisson',color='red',linestyle=(0, (5,↳
↳10)),linewidth=1)
diff_5000=gaussian(x,*popt)-poisson(x,*popt_p)
x_diff_5000=np.arange(int(n05[0]),int(n05[-8]),1)
diff_5000_g=-N[:-8]+gaussian(x_diff_5000,*popt)
diff_5000_p=-N[:-8]+poisson(x_diff_5000,*popt_p)

ax2.set_yscale('log')
ax1.legend()
# ax1.grid()
ax2.legend()
# ax2.grid()

plt.savefig(Ausgabe_path/'Histogramm0.1.png',format="png",bbox_inches='tight')
plt.tight_layout()
plt.show()

print("Gaussfit:")
print("A="+round_to_sigs(popt[0],np.sqrt(pcov[0][0]),r'')[3])
print("mu="+round_to_sigs(popt[1],np.sqrt(pcov[1][1]),r'\per{s'})[3])
print("sigma="+round_to_sigs(popt[2],np.sqrt(pcov[2][2]),r'\per{s'})[3])
print("Poissonfit:")

```

```

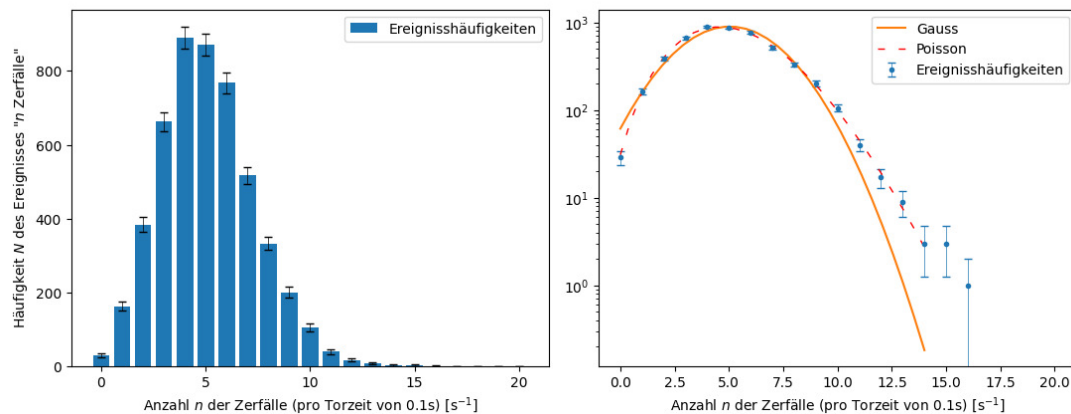
print(r"mu_p="+round_to_sigs(popt_p[1],np.sqrt(pcov_p[1][1]),r'\per\s')[3])
print(r"A_p="+round_to_sigs(popt_p[0],np.sqrt(pcov_p[0][0]),r'')[3])

#Gauss:
chi2_g=np.sum((gaussian(n05[:-8],*popt)-N[:-8])**2/fehlerN[:-8]**2)
dof_g=len(n05[:-8])-3 #dof:degrees of freedom, Freiheitsgrad
chi2_red_g=chi2_g/dof_g
print("chi2_g=", round_to_sigs(chi2_g)[0])
print("chi2_red_g=",round_to_sigs(chi2_red_g)[0])
#Poisson:
chi2_p=np.sum((poisson(n05[:-8],*popt_p)-N[:-8])**2/fehlerN[:-8]**2)
dof_p=len(n05[:-8])-2 #poisson hat nur 2 Parameter
chi2_red_p=chi2_p/dof_p
print("chi2_p=", round_to_sigs(chi2_p)[0])
print("chi2_red_p=",round_to_sigs(chi2_red_p)[0])

from scipy.stats import chi2
#Gauss:
prob_g=round(1-chi2.cdf(chi2_g,dof_g),2)*100
#Poisson:
prob_p=round(1-chi2.cdf(chi2_p,dof_p),2)*100
print("Wahrscheinlichkeit Gauss=", prob_g,"%")
print("Wahrscheinlichkeit Poisson=", prob_p,"%")

```

Statistik des radioaktiven Zerfalls: 5000 Messungen bei Torzeit 0.1s



Gaussfit:
 $A = 4900(80)$
 $\mu = 5.04(0.04) \text{ s}^{-1}$
 $\sigma = 2.174(0.025) \text{ s}^{-1}$
Poissonfit:
 $\mu_p = 5.08(0.04) \text{ s}^{-1}$

```

A_p=\num{4990(80)}
chi2_g= 120.0
chi2_red_g= 12.0
chi2_p= 6.0
chi2_red_p= 0.6
Wahrscheinlichkeit Gauss= 0.0 %
Wahrscheinlichkeit Poisson= 85.0 %

USELESS

```

```

[268]: plt.close('all')
x_2000=np.linspace(30,110, 100)
# plt.plot(x_2000,diff_2000,label='G-P 2000')
# plt.plot(x,diff_5000,label='G-P 5000')

plt.plot(x_diff_5000,diff_5000_g,label=r'5000: $N_{\text{exp}}-_{\text{G}}\{\mu,\sigma\}(n)$')
plt.hlines(np.
    ↪mean(diff_5000_g),x_diff_5000[0],x_diff_5000[-1],linestyle='--',color="C0",label='Mean')

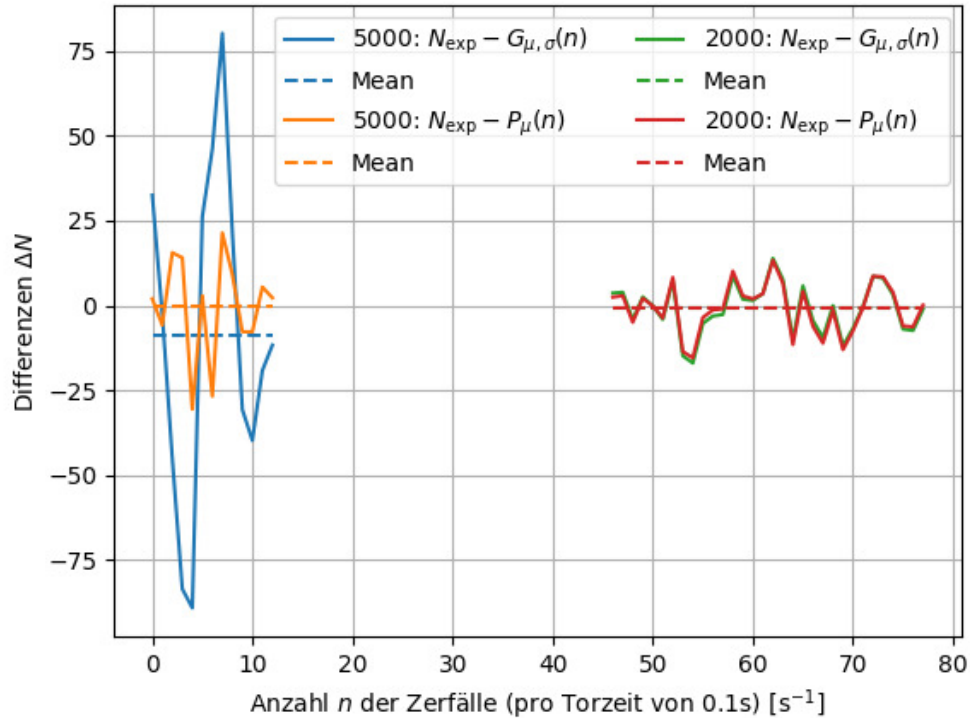
plt.plot(x_diff_5000,diff_5000_p,label=r'5000: $N_{\text{exp}}- P_{\mu}(n)$')
plt.hlines(np.
    ↪mean(diff_5000_p),x_diff_5000[0],x_diff_5000[-1],linestyle='--',color="C1",label='Mean')

plt.plot(x_diff_2000,diff_2000_g,label=r'2000: $N_{\text{exp}}-_{\text{G}}\{\mu,\sigma\}(n)$')
plt.hlines(np.
    ↪mean(diff_2000_g),x_diff_2000[0],x_diff_2000[-1],linestyle='--',color="C2",label='Mean')

plt.plot(x_diff_2000,diff_2000_p,label=r'2000: $N_{\text{exp}}- P_{\mu}(n)$')
plt.hlines(np.
    ↪mean(diff_2000_p),x_diff_2000[0],x_diff_2000[-1],linestyle='--',color="C3",label='Mean')

plt.xlabel('Anzahl $n$ der Zerfälle (pro Torzeit von 0.1s) [s$^{-1}$]')
plt.ylabel(r'Differenzen $\Delta N$')
plt.grid()
plt.legend(ncol=2)
plt.savefig(Ausgabe_path/'Abweichungen_dummd.
    ↪png',format="png",bbox_inches='tight')
plt.show()

```



```
[265]: plt.close('all')
x_2000=np.linspace(30,110, 100)
# plt.plot(x_2000,diff_2000,label='G-P 2000')
# plt.plot(x,diff_5000,label='G-P 5000')

plt.plot(x_diff_5000,(diff_5000_g/fehlerN5000)**2,label=r'5000: $(N_{\text{exp}}-\square$
    ↪G(n;\mu,\sigma)/\Delta N_{\text{exp}})^2$')
# plt.hlines(np.sum((diff_5000_g/
    ↪fehlerN5000)**2),x_diff_5000[0],x_diff_5000[-1],linestyle='--',color="C0",label='Mean')
print(np.sum((diff_5000_g/fehlerN5000)**2))
plt.plot(x_diff_5000,(diff_5000_p/fehlerN5000)**2,label=r'5000: $(N_{\text{exp}}-\square$
    ↪P(n;\mu,\sigma)/\Delta N_{\text{exp}})^2$')
# plt.hlines(np.sum((diff_5000_p/
    ↪fehlerN5000)**2),x_diff_5000[0],x_diff_5000[-1],linestyle='--',color="C1",label='Mean')
print(np.sum((diff_5000_p/fehlerN5000)**2))

plt.plot(x_diff_2000,(diff_2000_g/fehlerN2000)**2,label=r'2000: $(N_{\text{exp}}-\square$
    ↪G(n;\mu,\sigma)/\Delta N_{\text{exp}})^2$')
# plt.hlines(np.sum((diff_2000_g/
    ↪fehlerN2000)**2),x_diff_2000[0],x_diff_2000[-1],linestyle='--',color="C2",label='Mean')
```

```

print(np.sum((diff_2000_g/fehlerN2000)**2))

plt.plot(x_diff_2000,(diff_2000_p/fehlerN2000)**2,label=r'2000: $(N_{\text{exp}}-\mu-P(n;\mu,\sigma)/\Delta N_{\text{exp}})^2$')
# plt.hlines(np.sum((diff_2000_p/fehlerN2000)**2),x_diff_2000[0],x_diff_2000[-1],linestyle='--',color="C3",label='Mean')
print(np.sum((diff_2000_p/fehlerN2000)**2))

plt.xlabel('Anzahl $n$ der Zerfälle (pro Torzeit) [s$^{-1}$]')
plt.ylabel(r'$\chi^2_i$')
plt.grid()
plt.legend()
plt.savefig(Ausgabe_path/'Abweichungen.png',format="png",bbox_inches='tight')
plt.show()

```

115.23120822982594
 6.267113003266993
 30.31785092775567
 28.936992832110867

